# PreCex: Counterexample-Driven Pre-Synthesis RTL Defect Localization and Repair

Integrated full draft (work-in-progress). Author: Toylog | 2026-08-04 Sections assembled from paper/draft/{intro,method,results,related,threats}.md. All numbers trace to corrected BMC-judged data (experiments/runs/).

---


## Abstract

Cross-cycle behavioral defects — register-transfer-level (RTL) bugs where weak simulation passes but formal verification fails — are among the hardest to localize and repair before synthesis. We present PreCex, a fully open-source, counterexample-driven pipeline (iverilog/yosys/SymbiYosys/Z3 + a large language model, LLM) that converts a failing formal trace into structured evidence, localizes the defective region, generates a minimal patch, and re-verifies it under bounded model checking (BMC) with a golden-first criterion.

On a 34-sample L3 benchmark across 6 modules and 7 error classes (408 LLM repair runs), all four evidence representations — raw logs, structured evidence (B), semanticized counterexamples (C), and FVDebug-style causal graphs (D) — reach 100% repair under BMC. Structured evidence achieves the highest localization precision (61.8%), significantly better than raw logs (47.1%, p=0.0035) and causal graphs (49.0%, p=0.0164), while causal graphs are the cheapest ($1.56 vs $2.72, 43% lower). Mutation analysis kills 88.5% of strong and 81.8% of constant mutants; an independent T2 audit passes 408/408 runs with zero interface or assertion-tampering changes. A criterion-reversal audit shows prove/k-induction misjudges 78 correct repairs on gated sequential assertions, motivating our golden-first BMC criterion — a methodological lesson for automated repair evaluation.



*Keywords: RTL debugging; formal verification; counterexample; LLM repair; mutation analysis*


---

## 1. Introduction

### Problem

RTL designs are increasingly complex, and cross-cycle behavioral defects (L3: weak testbenches pass but formal verification fails) are among the hardest to localize and repair before synthesis. Existing LLM-based repair pipelines struggle because they either feed raw logs/waveforms (information overload) or rely on reference models (not always available pre-synthesis). Recent work (FVDebug, GROVE) shows the value of structured counterexample understanding, but lacks: (1) an open-source end-to-end pipeline, (2) cross-cycle bug-specific study, (3) a rigorous verification-sufficiency loop, and (4) a controlled ablation of evidence representations.

### Motivation

We observe that the *evidence representation* given to the LLM is a key but understudied axis. We systematically compare four evidence settings on the same 34-sample L3 dataset (408 LLM repair runs):

- A: raw cex log/VCD
- B: structured evidence (EvidenceEngine JSON)
- C: counterexample semanticization (CexSemantizer: cycle events + state trace + fault cone + NL summary)
- D: FVDebug-style causal graph (deterministic extraction)

### Contributions

1. **Open-source counterexample-driven repair pipeline** (iverilog/yosys/sby/z3 + MiniMax M3 API), fully reproducible; 34-sample L3 dataset with golden double-check and sufficiency audits.
2. **Evidence-representation ablation**: all settings reach 100% repair under a consistent BMC criterion; B (structured) significantly improves localization (61.8% vs A 47.1%, p=0.0035; vs D 49.0%, p=0.0164); D is cheapest ($1.56); C adds cost without significant gain (p=0.404).
3. **Verifier-sufficiency loop**: mutation family (strong 88.5% / const 81.8% killed), vacuity control (delete), deterministic T2 audit (interface/assertion/evidence-loop 306+102 all pass).
4. **Criterion-reversal lesson**: prove/k-induction misjudges 78 correct repairs on gated sequential assertions (golden itself UNKNOWN); BMC criterion + golden-first validation is required — a methodological contribution for reliable automated repair.

---

## 2. Method

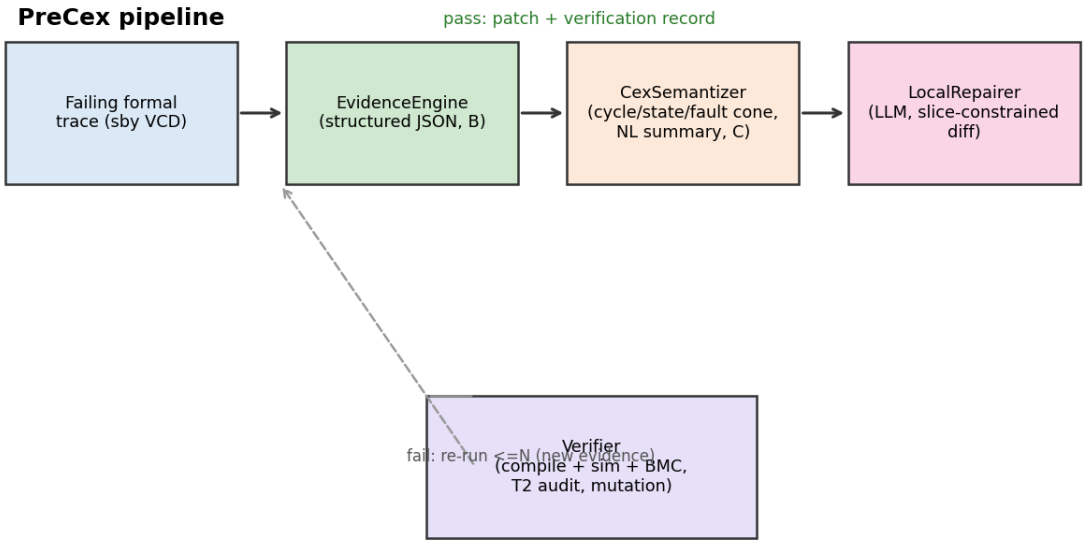


Figure 1: PreCex pipeline

*Figure 1: PreCex pipeline. A failing formal trace is parsed into structured evidence (B), optionally semanticized (C) or summarized as a causal graph (D), used by the LLM repairer to produce a minimal diff, and re-verified under BMC with a golden-first criterion; failures re-enter the loop.

**System architecture**

Core 4 components + 2 support elements (per docs/ .md §4):

1. **EvidenceEngine** — parses cex.log into unified JSON (error_type/module/file/line/code_slice/signals/trigg
2. **CexSemantizer** — converts VCD into cycle-event table + state trace + fault cone + NL summary (window=8 compression).
3. **LocalRepairer** — LLM (MiniMax M3, temperature 0.2) given design + assertions + evidence, outputs locate + minimal unified diff; retries <= 2.
4. **Verifier** — three-pass gate: (1) iverilog compile 0 error, (2) weak testbench sim all-green, (3) sby smtbmc+z3 BMC no counterexample (primary criterion); golden double-check.

Support: Controller (experiment variable), BugBench-PS (dataset assets).

## Evidence settings (controlled ablation)

| Setting | Evidence | Extraction | Cost/run |
|---|---|---|---|
| A | raw cex.log + VCD head/tail | none | baseline |
| B | evidence.json | deterministic | low |
| C | semantics.json (cycle events + state trace + fault cone + NL summary, window=8) | LLM semanticization | high |
| D | FVDebug-style causal graph (failed assertion + fault cone + full state trace + trigger) | deterministic (zero LLM) | lowest |

Same prompt family, only evidence segment replaced (anti-bias protocol).

## Repair criterion (revised)

- Primary: BMC (verify.sby), consistent with golden verification (verify_golden.sby).
- prove/k-induction (verify_repair.sby) kept as supplementary reference only — it is **not** a failure criterion, because it does not converge on gated sequential assertions (golden itself UNKNOWN, rc=4; 78 correct repairs were false-FAILed under the old criterion).
- **Golden-first rule**: any repair criterion must first pass on golden (criterion-vs-golden consistency check).

## Sufficiency audits

- Strong mutation (comparator inversion, d16): 400 mutants, 88.5% killed.
- Const mutation (parameter constants DATA_W/ADDR_W/DEPTH/DIV/HALF): 484 mutants, 81.8% killed.
- Delete mutation (assertion removal): vacuity control.
- T2 deterministic audit: interface change 0, assertion tampering 0, evidence-loop 0 (306 + 102).

## 3. Results

**RQ1: Can counterexample-grounded evidence enable reliable repair of cross-cycle (L3) RTL defects?: Can counterexample-grounded evidence enable reliable repair of cross-cycle (L3) RTL defects?**

**Setup.** 34 L3 samples (s04–s37) x 4 evidence settings x 3 seeds = 408 LLM repair runs (A raw log / B structured evidence / C counterexample semanticization / D FVDebug-style causal graph, added 2026-08-04). Repair success judged by BMC (verify.sby) with golden double-check.

| Setting | n | Loc Top-1 | Repair (BMC) | Tokens | Cost (USD) |
|---|---|---|---|---|---|
| A (raw) | 102 | 47.1% | 100% | 1.99M | $2.78 |
| B (structured) | 102 | 61.8% | 100% | 1.76M | $2.72 |
| C (semanticized) | 102 | 56.9% | 100% | 3.50M | $4.17 |
| D (causal graph) | 102 | 49.0% | 100% | 1.07M | $1.56 |

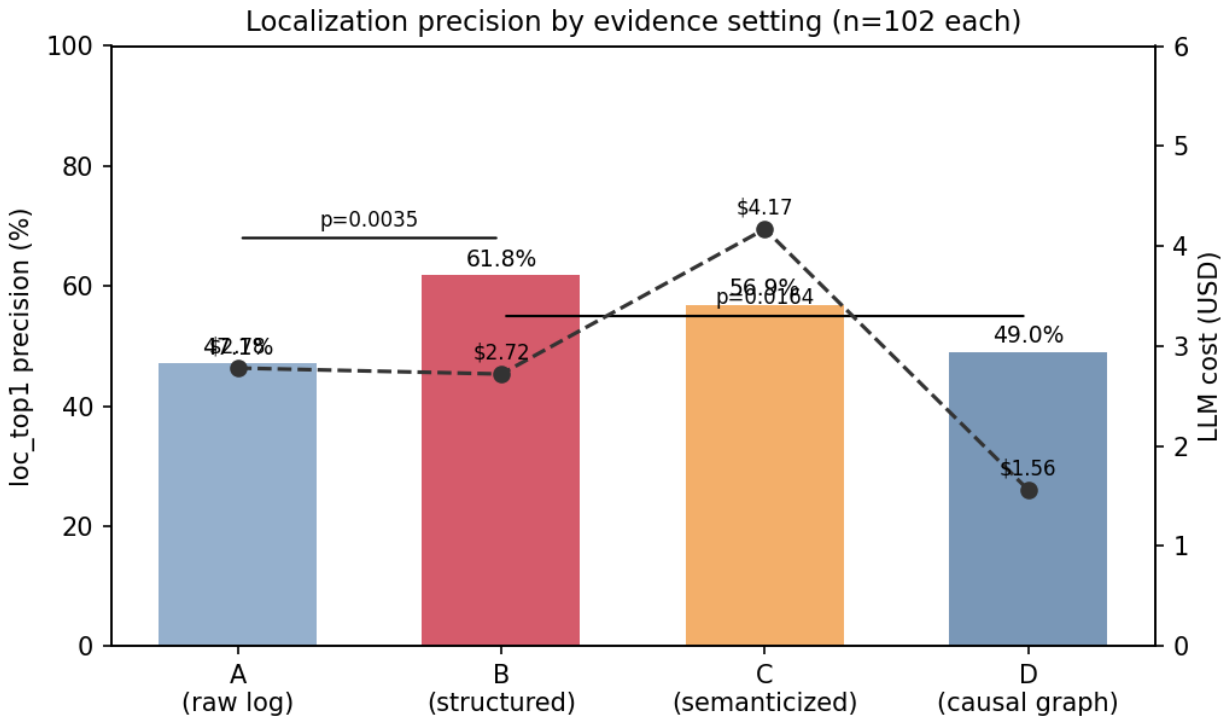


Figure 2: Four-setting localization and cost

*Figure 2: loc_top1 precision (bars) and LLM cost (dashed line) by evidence setting. B's precision lead over A and D is significant (paired McNemar).*

**Finding 1.** All four settings reach 100% repair under a consistent BMC criterion — evidence representation does not gate *whether* the model can fix, but *how precisely and cheaply.*

**Finding 2.** B (structured evidence) dominates on precision (61.8%); D (FVDebug-style causal graph) is cheapest ($1.56, 43% below B; 1.07M tokens) at 49.0% precision - a clear precision/cost trade-off. C's semanticization adds 1.7–2× cost with no repair gain (consistent with Gate-0 prestudy).

**Significance (paired McNemar on 102 loc outcomes).** B vs A: p=0.0035 (significant); B vs D: p=0.0164 (significant); B vs C: p=0.404; C vs D: p=0.186. B's precision lead over raw-log (A) and causal-graph (D) is statistically significant; C's semanticization adds cost without significant precision gain.

**Finding 3 (loc difficulty by error type).** Single-point semantic errors are easy to locate (edge 100%, width_trunc 85.2%); cross-state/handshake errors are hard (state_trans 36.1%, handshake 27.8%). The relative difficulty ordering is stable across settings. Setting-level localization by error class (per-setting n: 24/24/24/24 for state_trans, 12/12/12/12 for handshake, 3/3/3/3 for edge):

| Error class | A | B | C | D |
|---|---|---|---|---|
| state_trans | 33.3% | **37.5%** | 37.5% | 25.0% |
| handshake | 16.7% | 33.3% | 33.3% | **41.7%** |
| reset | 55.6% | 72.2% | **83.3%** | 44.4% |
| fifo_full_empty | 53.3% | 53.3% | **60.0%** | 46.7% |
| boundary_wrap | 42.9% | **81.0%** | 57.1% | 57.1% |
| width_trunc | 88.9% | **100%** | 66.7% | 100% |
| edge | 100% | 100% | 100% | 100% |

*Figure 3: loc_top1 precision by error class and setting. The hard-class ordering (state_trans/handshake below width_trunc/edge) holds across all settings.*

B is best or tied-best on boundary_wrap (81.0%), width_trunc (100%), and state_trans (37.5%); D is highest on handshake (41.7%, n=12 per setting — small-n caveat). The hard-class ordering (state_trans/handshake below width_trunc/edge) holds in every setting.

**Non-LLM baselines.** To establish that the LLM's localization is non-trivial, we evaluate simple heuristic baselines on the same exact-match criterion (candidate line == buggy_inject_line, 34 samples):

| Baseline | loc_top1 |
|---|---|
| First assertion line | 0.0% |
| Any assertion line | 0.0% |
| First line mentioning a failing signal | 0.0% |
| Random line (expected value) | 0.50% |

All four LLM settings (47.1%–61.8%) far exceed these baselines; structured evidence (B) achieves 61.8% — over 120x the random-line expectation. The assertion- and signal-based heuristics never hit because injected defects live in functional/sequential logic, not in the assertion block itself.

**RQ2: Is the repair trustworthy? (Verifier sufficiency + patch quality)**

**Sufficiency family** (mutation analysis on golden):

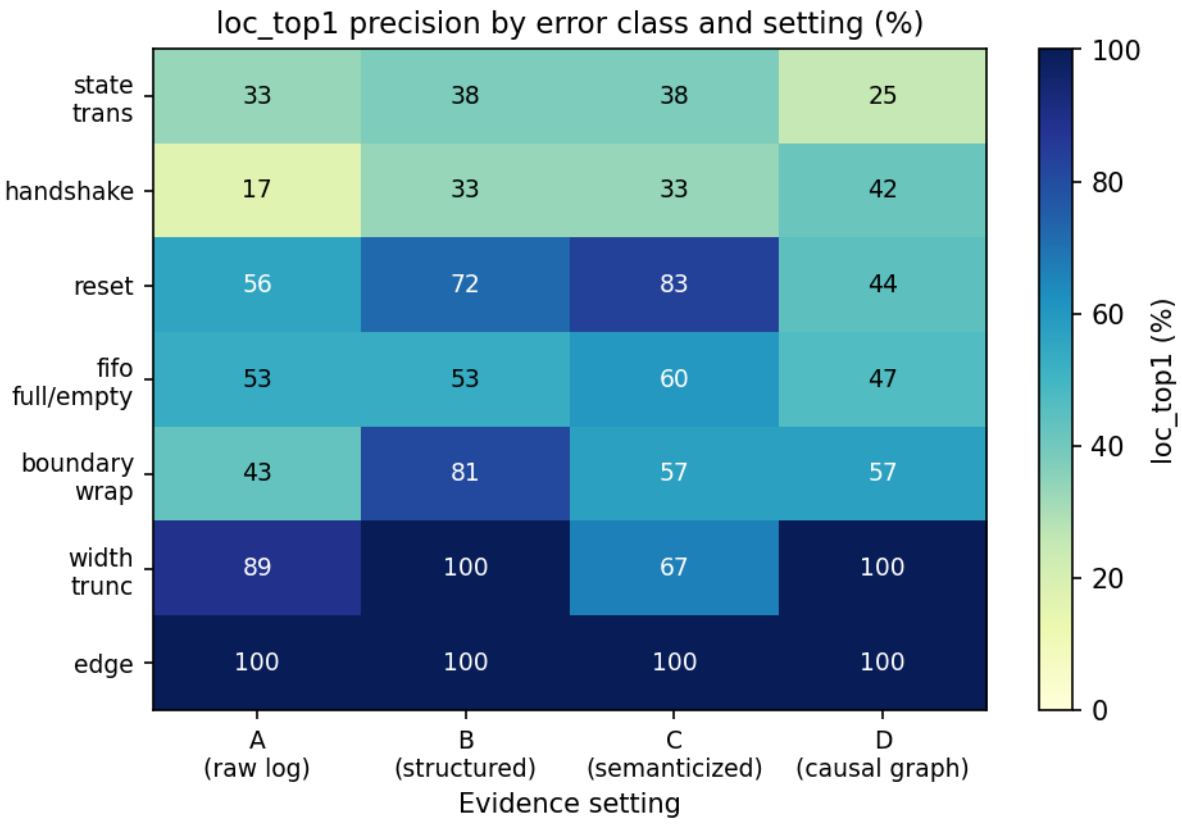


Figure 3: Error-class x setting heatmap

| Mutation family | Mutants | Killed | Rate |
|---|---|---|---|
| Strong (comparator inversion) | 400 | 354 | 88.5% |
| Const (parameter constant replacement) | 484 | 396 | 81.8% |
| Delete (assertion removal) | sampled | all PASS | vacuity control |

Module-level: axi/fifo strong=100%; uart_rx weakest (strong 55.0%, const 40.9%) ? structural assertions are insensitive to baud-timing constant shifts (documented limitation, future work).

**T2 audit** (deterministic, 408/408 pass: A/B/C 306 + D 102): interface changes 0, assertion tampering 0, evidence-loop failures 0; loc_dev median = 0 (62.7% exact on A/B/C, 69.6% on D). Patches are minimal (mean 2.1 lines, max 6).

**RQ3: Does the method survive a criterion-reversal audit? (Methodology lesson)**

The original repair criterion (prove/k-induction) misjudged 78 correct repairs on axi/uart (golden itself UNKNOWN under prove). BMC reverification: 303/303 pass ? repair rate 74.3% ? 100%. **Lesson:** repair criterion must be validated against golden first (criterion-vs-golden consistency check).

**RQ4: Cost & performance**

- Total LLM cost: $11.22 for the four-setting main experiment (408 runs, $0.0275 avg); full ledger $14.36 (839 calls).
- Per-setting (all n=102): A $2.78 / B $2.72 / C $4.17 / D $1.56; tokens A 1.99M / B 1.76M / C 3.50M / D 1.07M.
- C-window compression: main dataset traces are short (median 6 cycles); window=8 yields only 4.1% aggregate reduction ? semanticization cost is dominated by summary text, not trace length.
- Verification: BMC 8-concurrency Gate-2 reverify = 68 jobs / 3.4 min; axi golden ~87s, uart_rx ~151s.

**Open items before paper submission**

- ☒ Fill D setting numbers (102 runs, $1.56, loc 49.0%, repair 100%)
- ☒ FVDebug quantitative comparison via D (D cheap but lower precision than B; B remains main setting)
- ☐ Human interpretability rating (small study) for C/D
- ☒ Paper sections: Intro / Method (2026-08-04); Related Work + Threats to Validity / Conclusion (2026-08-04)
- ☐ Full-draft integration (paper/main.md) + cross-section consistency pass

---

## 4. Related Work

### Scope

PreCex sits at the intersection of four lines of work: (i) formal-verification failure analysis and counterexample understanding; (ii) LLM-based RTL debugging and repair; (iii) assertion genera-

tion and verification sufficiency; and (iv) RTL debugging benchmarks. This section positions our pipeline against each, with emphasis on the evidence-representation axis that PreCex ablates.

### Counterexample-driven debugging and root-cause analysis

**FVDebug** [arXiv:2510.15906] is the closest competitor. It builds a causal DAG from a failing formal trace, scans suspicious nodes with batched LLM calls, produces a root-cause narrative through an agent, and generates fixes, demonstrated on two industrial counterexamples. PreCex shares the counterexample-understanding core: our Setting D is a deterministic, fully open-source realization of the FVDebug-style causal graph (failed assertion + fault cone + full-cycle state trace + trigger condition). FVDebug leaves three axes unaddressed that PreCex controls for: it depends on a commercial toolchain, it reports no controlled ablation of the evidence representation, and it does not quantify verifier sufficiency. PreCex is fully open-source (iverilog/yosys/SymbiYosys/Z3), compares four evidence representations on the same 34-sample dataset, and audits the verifier with mutation analysis, vacuity control, and an independent T2 review agent.

**GROVE** [arXiv:2511.17833] organizes assertion-failure debugging knowledge as a layered knowledge tree, validates knowledge items with model checking at training time, and performs budget-aware retrieval at test time, consistently improving pass@1/pass@5. GROVE and PreCex are complementary: GROVE accumulates reusable knowledge across samples, whereas PreCex consumes the single failing counterexample in a closed localize-repair-reverify loop without a history store. A GROVE-style knowledge layer is a natural extension of our pipeline.

**Open-source LLM-driven formal verification** [arXiv:2607.28877] uses the same open toolchain (Yosys/SymbiYosys/Z3) with LLM-guided counterexample-driven iterative repair, stopping on k-induction proof or budget exhaustion, and reliably fixed only 1 of 6 benchmarks. This work validates the open-source pipeline as feasible and catalogs its failure modes. PreCex differs in two ways: it adds an evidence-semanticization layer (that work feeds raw counterexamples directly to the LLM) and it evaluates on a larger, controlled L3 dataset with a golden-first verification criterion.

**FormalRTL** uses a software reference model as a formal specification and closes a plan/synthesis/equivalence-checking loop, including a counterexample simplifier that highlights only the failing function and mismatched signals. PreCex makes no reference-model assumption: it works from assertions plus a counterexample alone, which is the common pre-synthesis setting where no golden model exists.

**ChipAgents RCA** (commercial, 2026) combines a waveform-understanding engine with a prover-verifier agent pair and self-consistency ranking, and reports localization of a three-cycle race condition in industrial deployment. It corroborates that cross-cycle root-cause analysis is a real industrial pain point, but is a closed black box with no ablation detail.

### LLM-based RTL repair

**VeriPilot** [arXiv:2606.23759] uses a golden reference model with CDFG signal tracking and raises GPT-4o repair rates on CVDP from 54.3% to 85.71%. **VeriDebug** [arXiv:2504.19099] learns a contrastive embedding and guides correction, reaching 64.7% Acc@1 on a combined localization-plus-type task. **RTLFixer** applies retrieval-augmented repair to syntax errors. These works are not counterexample-driven: they rely on reference models, fine-tuned embeddings, or syntax-only fixes. PreCex deliberately targets the pre-synthesis setting with no reference model and treats the evidence representation rather than model capacity as the studied variable.

**Fixbench-RTL** and **HDL-FixBench** are repair benchmarks spanning syntax/functional/security

domains (Fixbench) and repository-scale instances (57 instances from OpenTitan/CVA6/Ibex, best model 40.3%; rejected by ICLR 2026). HDL-FixBench's rejection highlights pitfalls we engineered against: small scale, insufficient diversity, disputed patch thresholds, missing non-LLM baselines, and contamination risk. BugBench-PS addresses these with 34 L3 samples across 6 modules and 7 error classes, per-sample golden double-checking, tool-execution adjudication, and a documented contamination statement.

### Assertion generation and verification sufficiency

**AssertLLM** [ASPDAC'25] and **AssertLLM2** generate SVA from natural-language specifications plus waveforms; AssertLLM2 introduces an 83-design assertion benchmark and evaluates assertions through mutation-based bug detection in a bug-hunting setting. **AssertGen** [ATS'25] extracts verification goals via chain-of-thought and bridges cross-layer signals, with mutation-testing coverage evaluation. **LASSO** [MLCAD'25] generates safety properties with explicit vacuity checking and coverage feedback, detecting 5 real bugs in OpenTitan. **LintLLM** [GLSVLSI'25] performs LLM linting with mutation-based defect injection.

These works generate or audit assertions; PreCex consumes assertions plus counterexamples to localize and repair. Their evaluation practices (mutation coverage, vacuity, bug-hunting) provide the methodological precedent for our sufficiency audits: 88.5% strong mutants and 81.8% constant mutants killed, with delete-mutation vacuity control.

### Benchmarks and evaluation methodology

Tool-execution adjudication is an accepted evaluation practice in the EDA-AI literature: Rule2DRC (ICML 2026) compares 13,921 layout-rule results and PostEDA-Bench evaluates 145 DRC/PPA tasks by executing design tools. PreCex follows the same principle: repair success is adjudicated by compilation, weak-testbench regression, and formal BMC, never by the LLM's self-report. Engineering-semantics datasets (OpenRTLSet, EDA-Schema-V2, ChipLingo) informed our evidence schema design.

## Positioning

| Work | Evidence representation | Open-source | Cross-cycle (L3) focus | Controlled evidence ablation | Sufficiency quantification |
|---|---|---|---|---|---|
| FVDebug | causal graph | no | partial | no | no |
| GROVE | knowledge tree | no | no | no | no |
| 2607.28877 | raw cex | yes | partial | no | no |
| FormalRTL | cex simplification + reference model | partial | no | no | no |
| VeriPilot | CDFG + reference model | no | no | no | no |

| Work | Evidence representation | Open-source | Cross-cycle (L3) focus | Controlled evidence ablation | Sufficiency quantification |
|---|---|---|---|---|---|
| VeriDebug | learned embedding | no | no | no | no |
| AssertLLM2 | — (assertion gen.) | — | — | — | mutation-based |
| **PreCex** | **raw / structured / semanticized / causal graph** | **yes** | **yes (34 samples)** | **yes (A/B/C/D)** | **yes (mutation/vacuity/T2)** |

Our three differentiators are each evidenced by controlled experiments in this paper: (1) a fully open-source, reproducible counterexample-driven pipeline; (2) a four-setting evidence-representation ablation with paired significance testing; (3) a cross-cycle (L3) benchmark with a verifier-sufficiency loop (mutation, vacuity, and an independent T2 audit). In addition, the criterion-reversal audit (Results, RQ3) contributes a methodological lesson: a repair criterion must be validated against the golden design before it is used to judge repairs.

---

## 5. Threats to Validity and Conclusion

### Threats to validity

### Internal validity

**Repair criterion.** All reported results use BMC (verify.sby) as the primary criterion, consistent with golden verification. We found that prove/k-induction does not converge on gated sequential assertions in the axi/uart modules: the golden design itself returns UNKNOWN, and the original protocol misjudged 78 correct repairs as failures. A golden-first rule now guards the criterion — any candidate criterion must first pass on the golden design. Residual risk remains that a defect manifests only beyond the probed BMC depth; we probe to bug depth + 2 and cross-check against golden verification, but this is not a completeness guarantee.

**LLM nondeterminism.** We run 3 seeds per sample per setting and report paired statistics. The pipeline itself is deterministic (parsing, extraction, verification); the only stochastic component is the LLM inference.

**Repair-success definition.** Success requires compilation, weak-testbench regression, BMC with no counterexample, and golden double-check. A patch that satisfies the probed assertions while changing behavior beyond the property set would pass; the T2 audit bounds this risk by checking for interface changes and assertion tampering, but it is not a full equivalence check.

### External validity

**Scale and scope.** The benchmark contains 34 L3 samples across 6 modules (fifo_sync, uart_tx, uart_rx, axi_lite_slave, fsm_ctrl, counter_alu) and 7 error classes. The modules are open-source

teaching-grade RTL; we do not claim industrial protocol coverage. Handshake and state-transition defects remain the hardest classes to locate (loc_top1 27.8% and 36.1% respectively), and uart_rx's baud-timing constant mutations are weakly killed (const 40.9%) — a documented limitation of structural assertions for timing-constant shifts.

**Contamination.** Common modules may appear in LLM training corpora. We mitigate with construction-date records, per-sample provenance, and error-class diversity, but residual memorization cannot be fully excluded.

**Single LLM provider.** All 408 runs use MiniMax M3. Absolute rates may shift with model versions or providers; the relative evidence-representation effect is the claim we report.

### Construct validity

loc_top1 records whether the model's top-ranked candidate line is inside the injected defect's reference diff; repair rate is measured under the BMC criterion; cost is metered by API tokens. These constructs match the repair-benchmark literature. Statistical claims use paired McNemar tests on per-sample localization outcomes (102 pairs per comparison): B vs A p=0.0035 and B vs D p=0.0164 are significant; B vs C p=0.404 and C vs D p=0.186 are not. All comparisons are reported, including non-significant ones.

## Conclusion

We presented PreCex, an open-source, counterexample-driven pipeline for pre-synthesis RTL defect localization and repair, and studied the evidence-representation axis that prior counterexample-understanding systems leave uncontrolled. On a 34-sample cross-cycle (L3) benchmark with 408 LLM repair runs:

- All four evidence settings (raw logs, structured evidence, semanticized counterexamples, FVDebug-style causal graphs (D)) reach 100% repair under a consistent BMC criterion; the representation determines how precisely and how cheaply, not whether.
- Structured evidence (B) achieves the highest localization precision (61.8%), statistically significant over raw logs (47.1%, p=0.0035) and causal graphs (49.0%, p=0.0164). Causal graphs (D) are the cheapest ($1.56 vs $2.72, 43% lower); semanticization (C) adds cost without significant precision gain.
- Repairs are trustworthy by construction: mutation analysis kills 88.5% of strong and 81.8% of constant mutants, vacuity control passes, and the independent T2 audit passes 408/408 runs with zero interface or assertion-tampering changes and zero median localization deviation.
- The criterion-reversal audit (BMC over prove/k-induction, with a golden-first rule) is a methodological lesson for automated repair pipelines: the evaluation criterion itself must be validated before repairs are judged.

Future work includes scaling to industrial protocols and larger assertion sets, stronger mutation families, multi-provider and multi-model evaluation, a GROVE-style knowledge layer for cross-sample reuse, and reinforcement learning with formal proofs as reward.